Instead, you can create an instance of the `EmailServerSettings` object yourself and provide that to the container, as shown in the following listing. The container uses the preconstructed object whenever an instance of the `EmailServerSettings` object is required.

Listing 9.4 Providing an object instance when registering services

```
WebApplicationBuilder builder = WebApplication.CreateBuilder(args);

builder.Services.AddScoped<IEmailSender, EmailSender>();
builder.Services.AddScoped<NetworkClient>();
builder.Services.AddSingleton<MessageFactory>();
builder.Services.AddSingleton(
        new EmailServerSettings          ❶
        (                                ❶
            Host: "smtp.server.com",     ❶
            Port: 25                     ❶
        ));                              ❶


WebApplication app = builder.Build();

app.MapGet("/register/{username}", RegisterUser);

app.Run();
```

❶ This instance of EmailServerSettings will be used whenever an instance is required.

This code works fine if you want to have only a single instance of `EmailServerSettings` in your application; the same object will be shared everywhere. But what if you want to create a *new* object each time one is requested?

NOTE When the same object is used whenever it's requested, it's known as a *singleton*. If you create an object and pass it to the container, it's always registered as a singleton. You can also register any class using the `AddSingleton<T>()` method, and the container will use only one instance throughout your application. I discuss singletons and other lifetimes